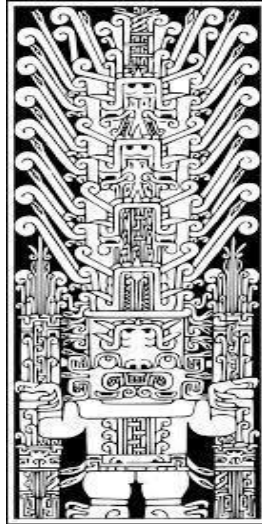


**UNIVERSIDAD NACIONAL FEDERICO VILLARREAL
FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS**



INFORME DE ATAQUE A UNA APLICACION WEB

OBJETIVO

American Pistacho Growers

AUTOR

Quispe Mescua George Angel

CURSO

Ciberseguridad

2026

1. Resumen

Se identificó y explotó una vulnerabilidad crítica de Inyección SQL en el panel de autenticación administrativa de una aplicación web desarrollada en PHP. La vulnerabilidad permitió eludir el mecanismo de inicio de sesión sin necesidad de credenciales válidas, lo cual representa un riesgo crítico para la confidencialidad, integridad y disponibilidad de la información gestionada por la aplicación.

El vector de entrada fue identificado inicialmente mediante técnicas de reconocimiento pasivo (Google Dorks), que permitieron localizar formularios de autenticación expuestos públicamente y, posteriormente, confirmar y explotar la vulnerabilidad de SQLi mediante pruebas manuales y automatizadas.

Se presenta un cuadro resumen de los hallazgos:

Hallazgo	Vector	Severidad	Impacto
Inyección SQL en formulario de autenticación	Parámetros 'usuario' / 'contraseña' del panel administrativo	Crítica	Acceso no autorizado a panel de administración
Exposición de paneles de administración indexados en buscadores	Google Dorks (intitle:/inurl:)	Alta	Superficie de ataque ampliada y reconocimiento facilitado
Mensajes de error explícitos en respuestas de la aplicación	Respuesta HTTP del formulario de login	Media	Facilita enumeración de estructura de BD
Ausencia de mecanismos de limitación de intentos	Endpoint de autenticación	Media	Permite ataques de fuerza bruta o inyección iterativa

2. Alcance de la evaluación

Cliente	American Pistachio Growers
Activo evaluado	http://carboncalculator.americanpistachios.org/admin/index.php
Tipo de evaluación	Black-box Web Application Penetration Test
Metodología	Módulo 15: SQL Injection

3. Metodología

La evaluación siguió las fases clásicas de un test de intrusión web, alineadas con el contenido del Módulo 15 de CEH:

1. Reconocimiento: Identificación de activos expuestos mediante operadores de búsqueda avanzada (Google Dorks).
2. Identificación de la vulnerabilidad: Análisis del comportamiento del formulario de autenticación ante entradas no esperadas.
3. Explotación: construcción de payloads de inyección SQL para eludir el mecanismo de autenticación.
4. Post-explotación: Determinación del alcance del acceso obtenido.
5. Documentación y reporte: Priorización de hallazgos según severidad y elaboración de recomendaciones de remediación.

3.1 Reconocimiento con Google Dorks

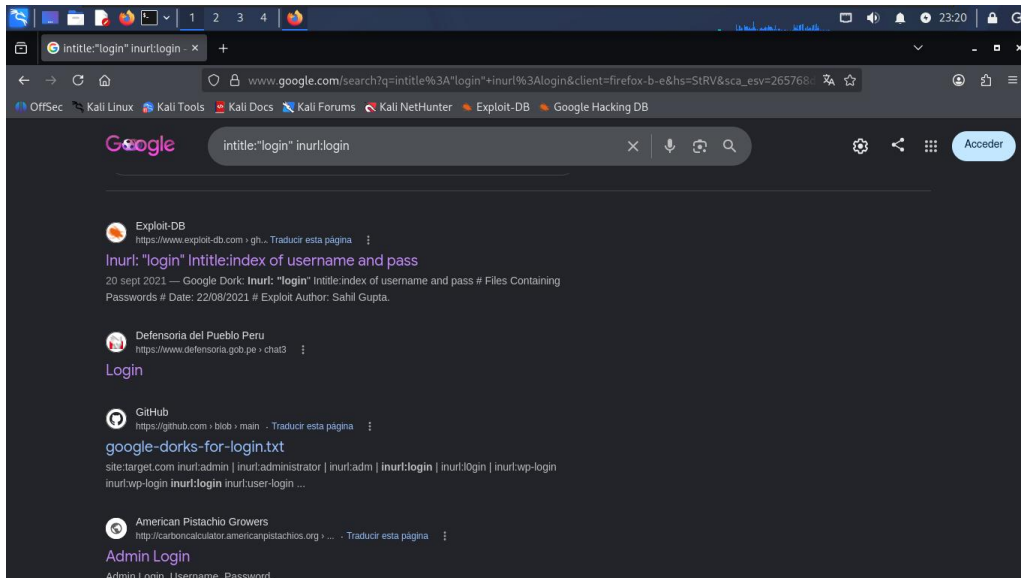
Se emplearon operadores de búsqueda avanzada de Google (Google Dorks) para identificar paneles de inicio de sesión potencialmente vulnerables expuestos en internet, sin interactuar directamente con los paneles en esta fase. Esta técnica corresponde a la fase de Footprinting.

```
intitle:"login" inurl:login
```

```
inurl:admin/index.php
```

```
intitle:"index of" "username" "password"
```

Este tipo de búsquedas permite localizar formularios de autenticación administrativa que los motores de búsqueda han indexado públicamente, reduciendo el esfuerzo de reconocimiento activo y, por lo tanto la probabilidad de detección temprana se incrementa.



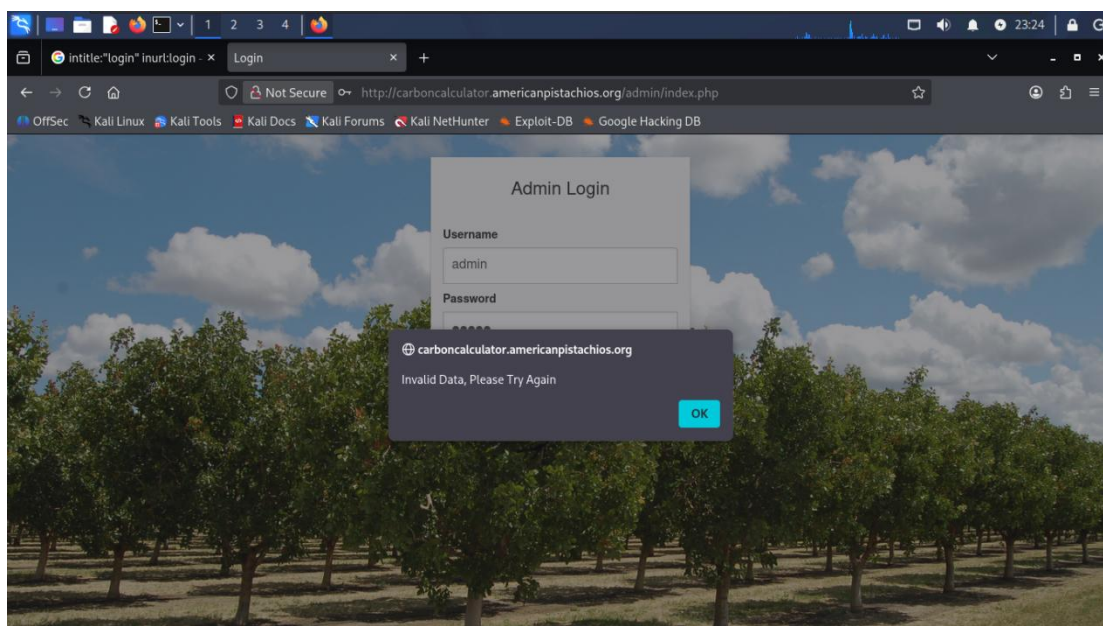
3.2 Identificación de la vulnerabilidad

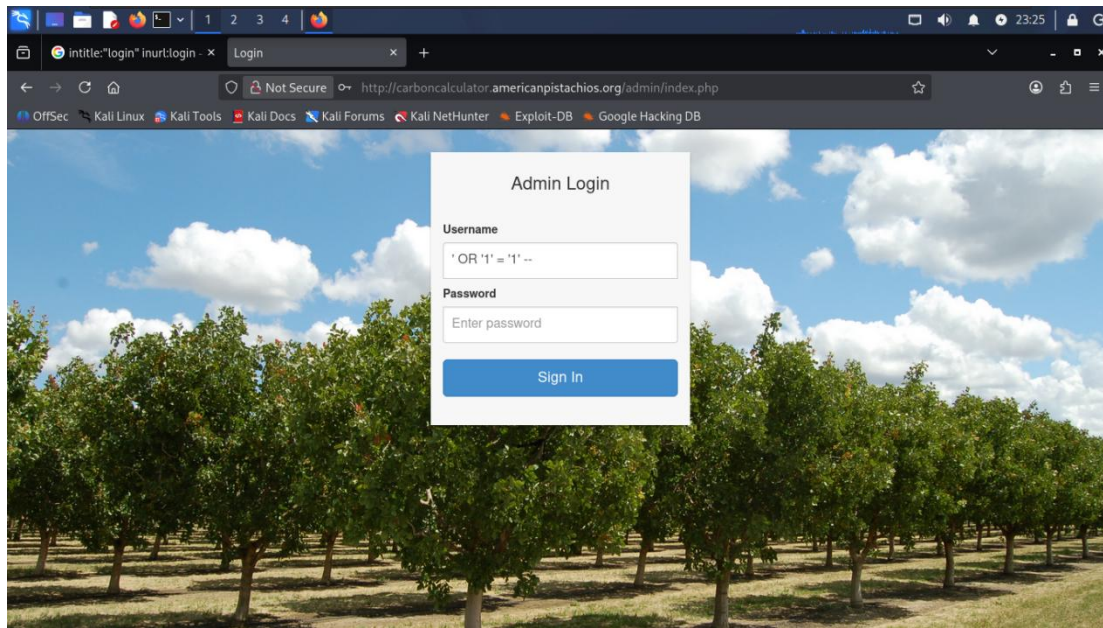
Tras acceder al panel de autenticación identificado como:

<http://carboncalculator.americanpistachios.org/admin/index.php>

Se observó un formulario estándar de usuario y contraseña desarrollado en PHP, sin protecciones visibles de tipo CAPTCHA ni rate limiting.

Se realizaron pruebas iniciales con caracteres especiales como comillas simples, comentarios SQL en los campos del formulario, obteniendo respuestas de error que sugerían la concatenación directa de la entrada del usuario dentro de la consulta SQL en el backend un indicador clásico de inyección SQL.





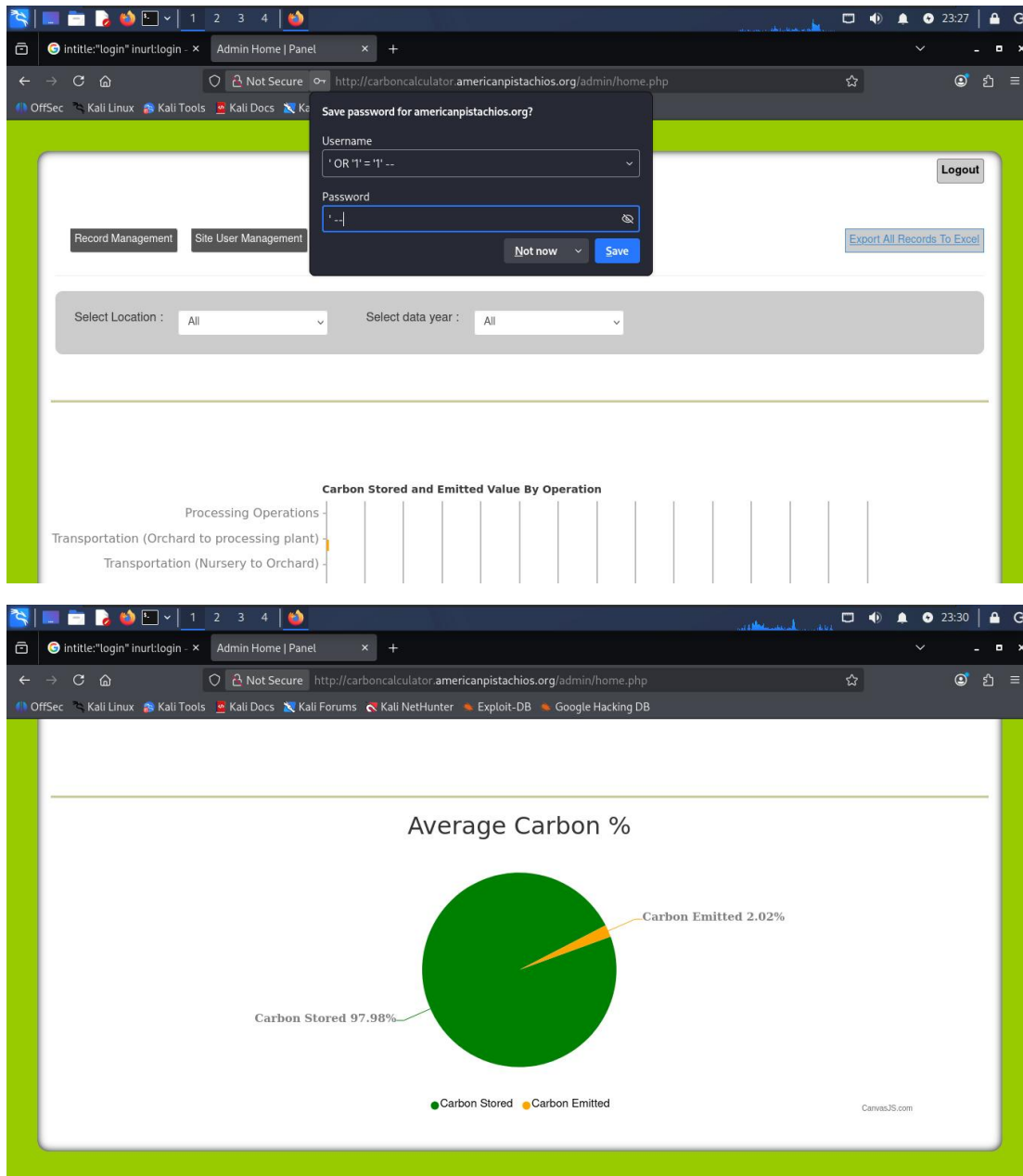
3.3 Explotación

Una vez confirmado el comportamiento de la página se procedió a emplear dos de los tipos “in-band SQL Injection”, siendo estos “Tautología” y “Comentarios al terminar la línea”

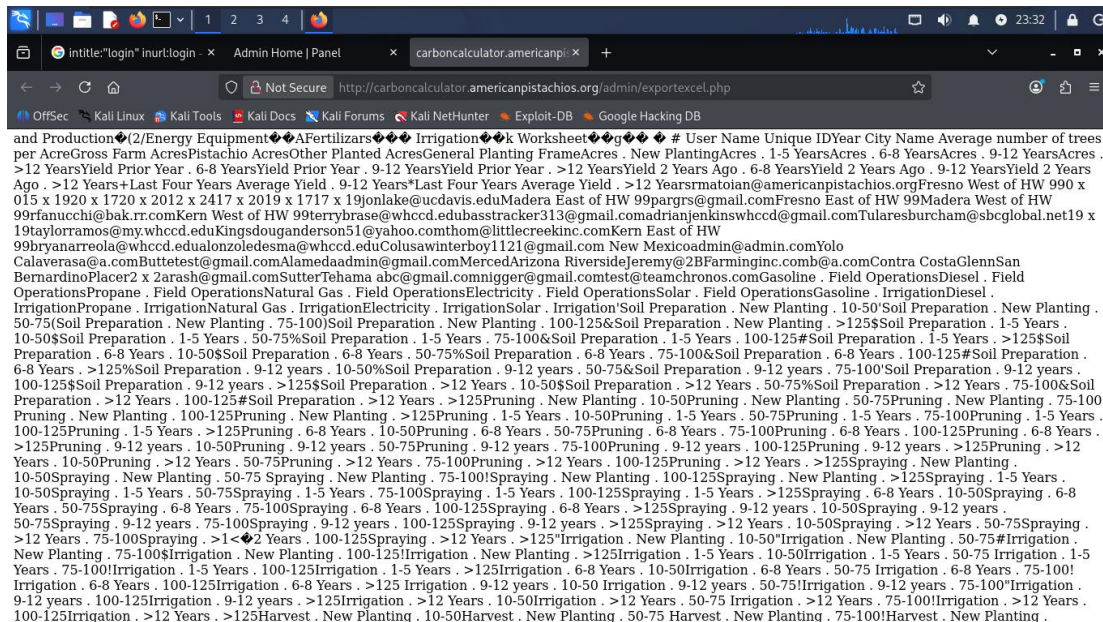
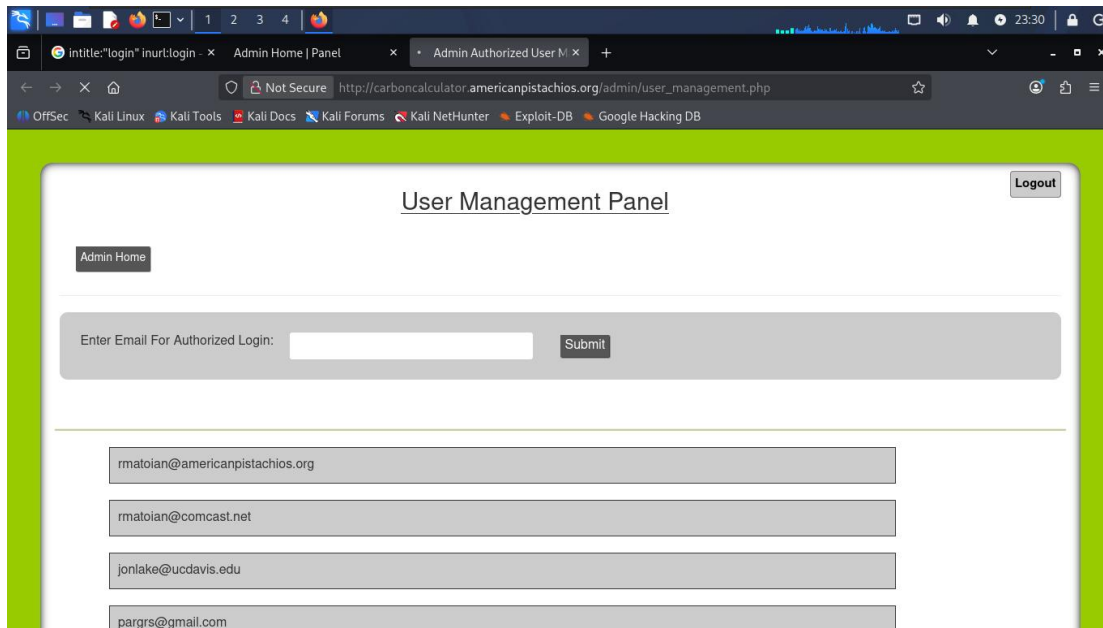
Usuario: admin' OR '1'='1' --

Contraseña: --

Esta entrada modifica la cláusula “WHERE” de la consulta SQL subyacente de forma que la condición evaluada siempre resulta verdadera, permitiendo el acceso sin necesidad de credenciales legítimas. Como resultado, se obtuvo acceso no autorizado al panel administrativo de la aplicación, confirmando la explotabilidad de la vulnerabilidad.



El hecho de que la página sea un sitio de administrador concede el acceso al modulo de administración de usuarios y exportación de la información principal de la página:



4. Hallazgos y Priorización

Los hallazgos se priorizan según su severidad y el impacto potencial sobre el negocio:

4.1 Hallazgo Crítico: Bypass de autenticación vía SQL Injection

Permite al atacante no autenticado obtener acceso administrativo completo a la aplicación, comprometiendo la confidencialidad e integridad de los datos almacenados.

4.2 Hallazgo Alto: Exposición de paneles administrativos en motores de búsqueda

La indexación pública de rutas administrativas amplía la superficie de ataque y facilita el reconocimiento por parte de actores maliciosos.

4.3 Hallazgo Medio: Mensajes de error verbosos

Las respuestas del servidor revelan información sobre la estructura interna de la consulta SQL, facilitando ataques de enumeración.

5.4 Hallazgo Medio: Ausencia de control de intentos de inicio de sesión

La falta de bloqueo temporal o CAPTCHA permite la ejecución de ataques automatizados ya sea por fuerza bruta o por inyección iterativa sin restricciones.

5. Impacto

- Acceso no autorizado a información confidencial almacenada en la base de datos como datos de usuarios, configuración del sistema, etc.
- Posibilidad de modificación o eliminación de registros.
- Riesgo de pivoteo hacia otros sistemas internos si la base de datos comparte credenciales o red con otros activos.
- Daño reputacional y posibles implicancias legales/regulatorias en caso de fuga de datos personales.

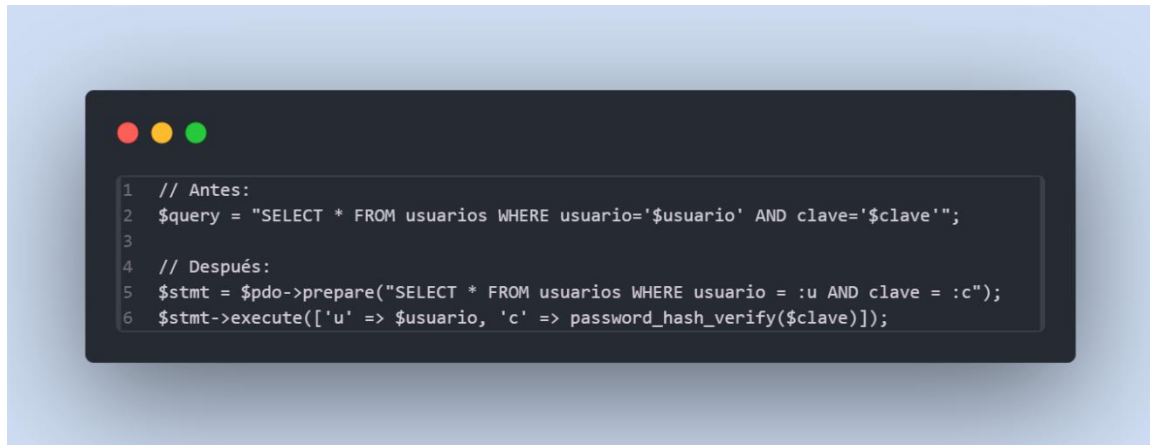
6. Recomendaciones

Se presentan las recomendaciones priorizadas para cada hallazgo identificado:

Vulnerabilidad	Recomendación	Prioridad
Inyección SQL	Uso de consultas parametrizadas (PDO o MySQLi) en lugar de concatenación de cadenas	Inmediata
Inyección SQL	Implementar un WAF (ModSecurity con OWASP CRS, Cloudflare, etc.) frente a la aplicación	Inmediata
Exposición en buscadores	Bloquear indexación de rutas administrativas y restricciones de red	Alta
Mensajes de error	Desactivar el modo de depuración (display_errors=Off) y usar páginas de error genéricas en producción	Media
Autenticación	Agregar autenticación multifactor y bloqueo temporal tras intentos fallidos	Media
Monitoreo	Centralizar logs y generar alertas ante patrones típicos de SQLi como: UNION, OR 1=1	Media

6.1 Consultas parametrizadas

La causa raíz del hallazgo crítico es la construcción de consultas SQL mediante concatenación de cadenas. La corrección recomendada consiste en migrar a consultas preparadas, donde la entrada del usuario nunca se interpreta como código SQL:

A screenshot of a code editor with a dark background and light blue lines. The code is written in PHP and shows the transition from a concatenated SQL query to a prepared statement. The code is as follows:

```
1 // Antes:
2 $query = "SELECT * FROM usuarios WHERE usuario='$usuario' AND clave='$clave'";
3
4 // Después:
5 $stmt = $pdo->prepare("SELECT * FROM usuarios WHERE usuario = :u AND clave = :c");
6 $stmt->execute(['u' => $usuario, 'c' => password_hash_verify($clave)]);
```

6.2 Controles complementarios

- Validación y sanitización de entradas en el servido.
- Implementación de un WAF con reglas específicas contra SQLi.
- Restricción de privilegios de la cuenta de base de datos utilizada por la aplicación.
- Desactivado de mensajes de error detallados en producción.
- Implementación de MFA para el panel administrativo.